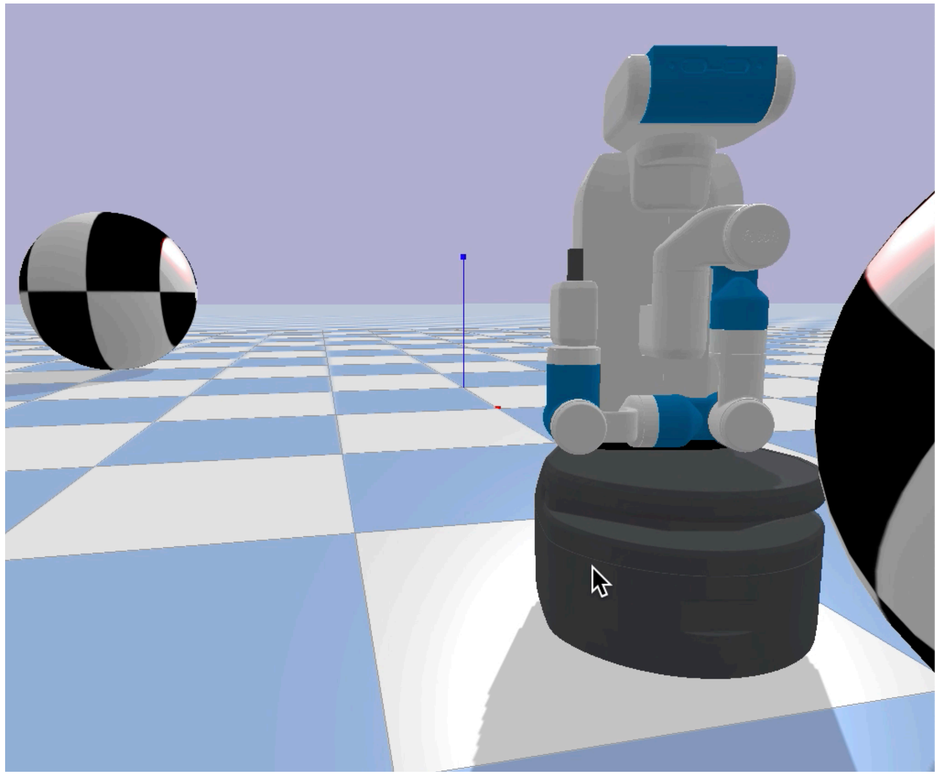
ROB-UY 3303

Robot Motion & Planning

Lecture 05A - Point Tracking



Agenda

1. Proportional Control
2. Linear Systems
3. Point Tracking

Agenda

1. **Proportional Control**
2. Linear Systems
3. Point Tracking

- Proportional Feedback Control – P Control
  - Uses the error between the desired and measured state to determine the control signal.

- If x_{des} is the desired state, and x is the actual state, we define the error as:

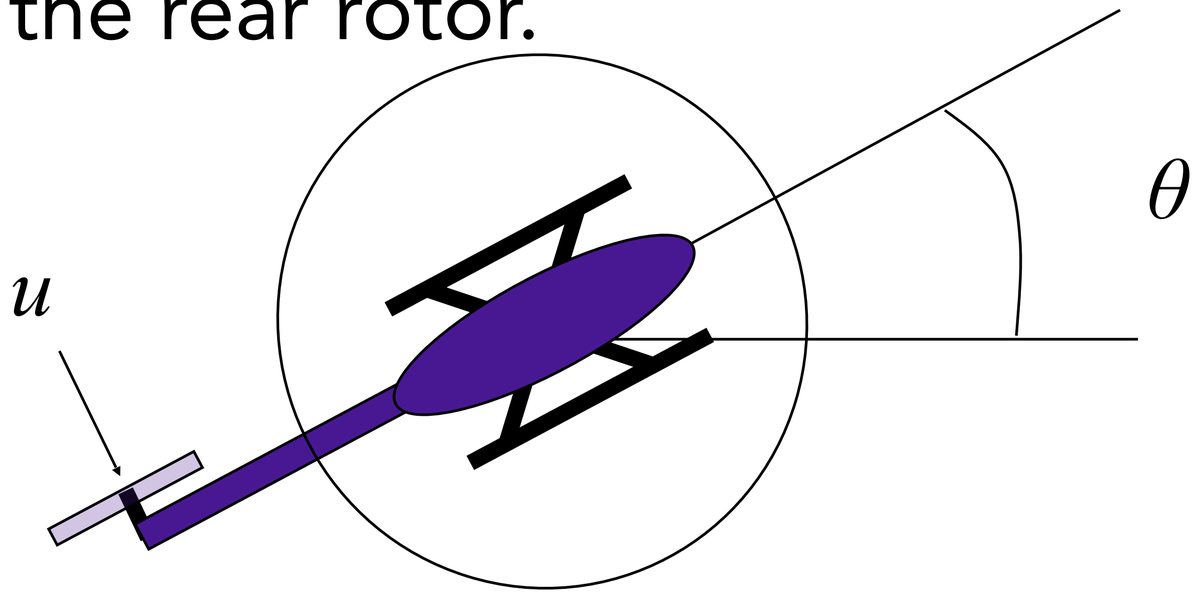
$$e = x_{des} - x$$

- The control signal u is calculated as

$$u = K_P e$$

where K_P is called the proportional gain.

- Example:
  - Consider the orientation control of an autonomous helicopter. Assume the orientation is completely controlled by the rear rotor.

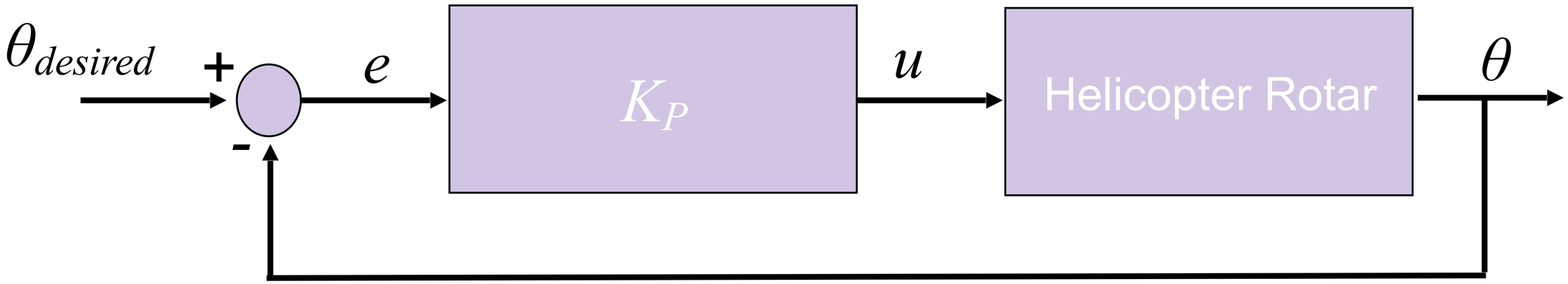


- The control signal u is calculated as

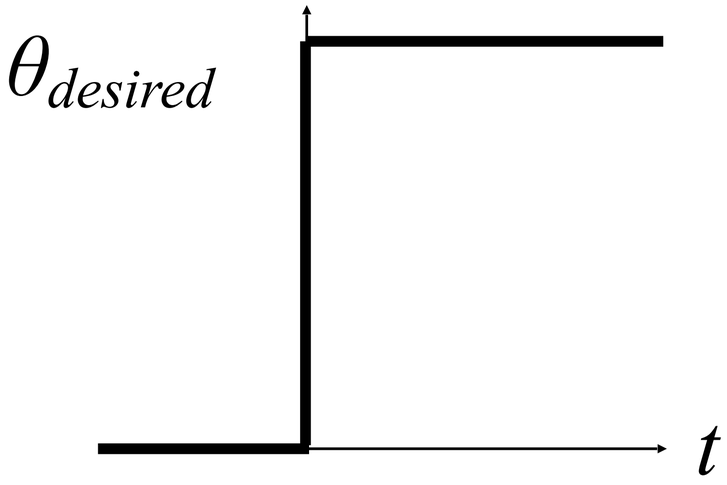
$$u = K_P(\theta_{desired} - \theta)$$

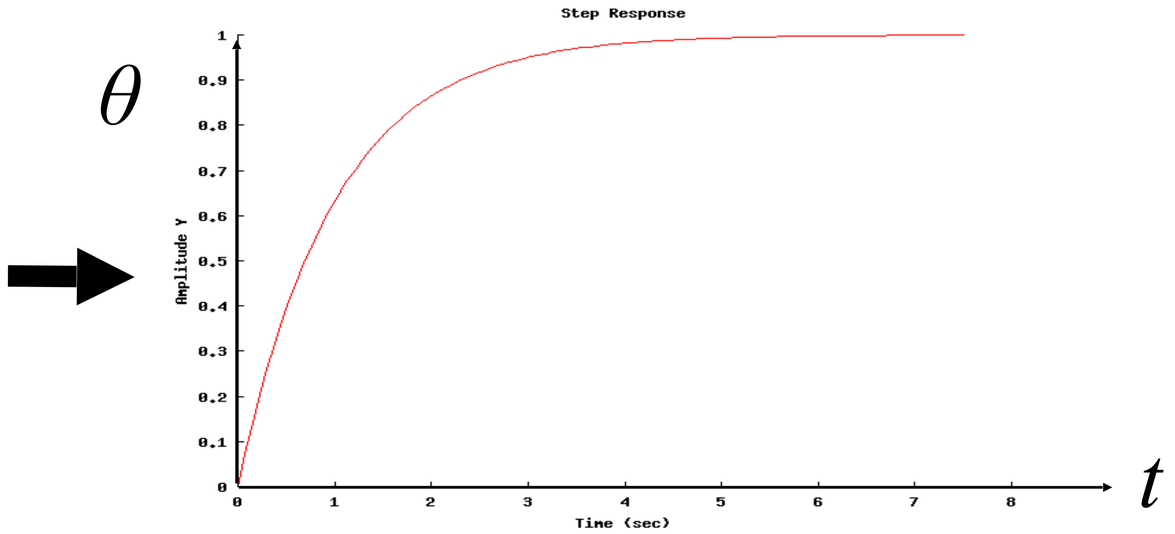
- Block Diagram:

$$u = K_P(\theta_{desired} - \theta)$$

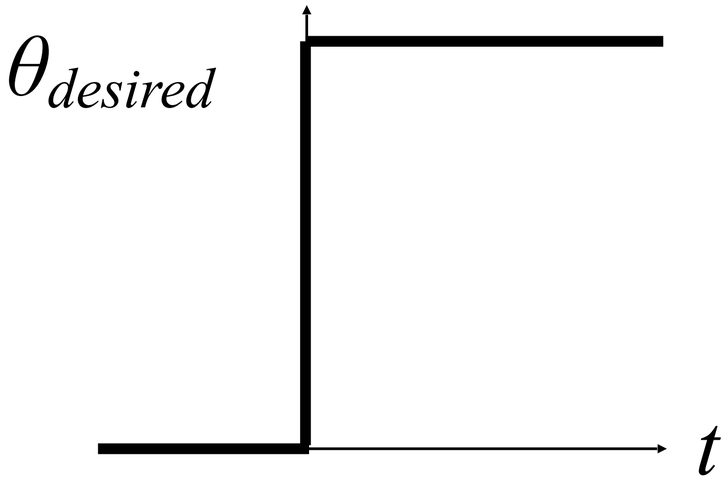


- Time Domain Response of step response
  - Step from $\theta_{desired} = 0$ to $\theta_{desired} = 1$.

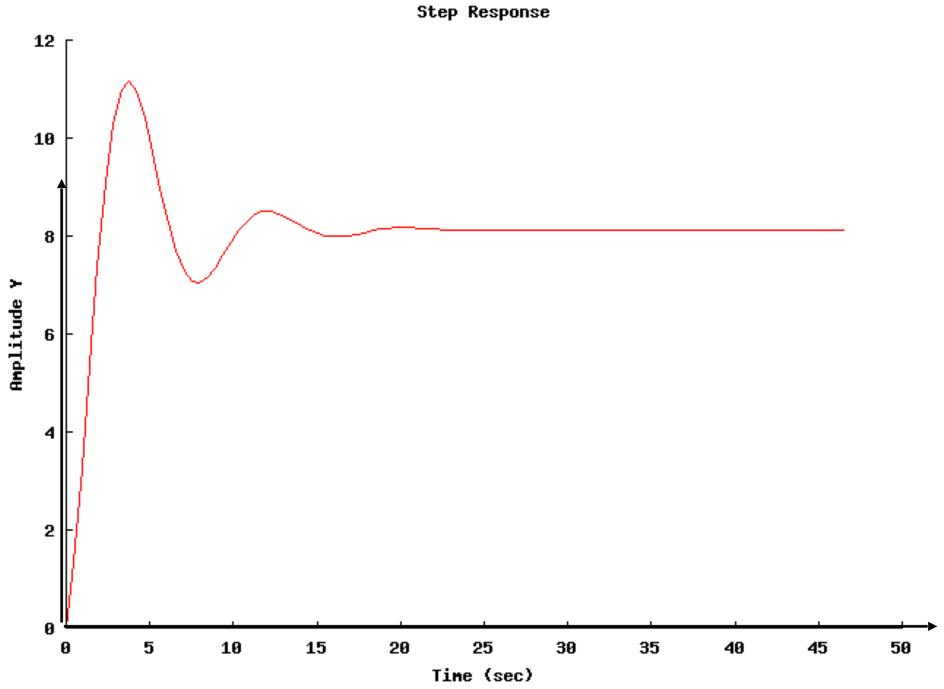




- Time Domain Response of step response
  - Step from $\theta_{desired} = 0$ to $\theta_{desired} = 8$.
  - Different dynamics in this case



θ



Agenda

1. Proportional Control
2. **Linear Systems**
3. Point Tracking

- Recall that the forward kinematics of a robot can be described as a linear differential equation.
  - E.g. a robot state x behaves as if it obeys the following equation:

$$\dot{x} = \frac{dx}{dt} = ax$$

where a is a constant

- One can observe that the solution to the equation:

$$\dot{x}(t) = ax(t)$$

is

$$x(t) = x_0 e^{at}$$

where x_0 is the initial state

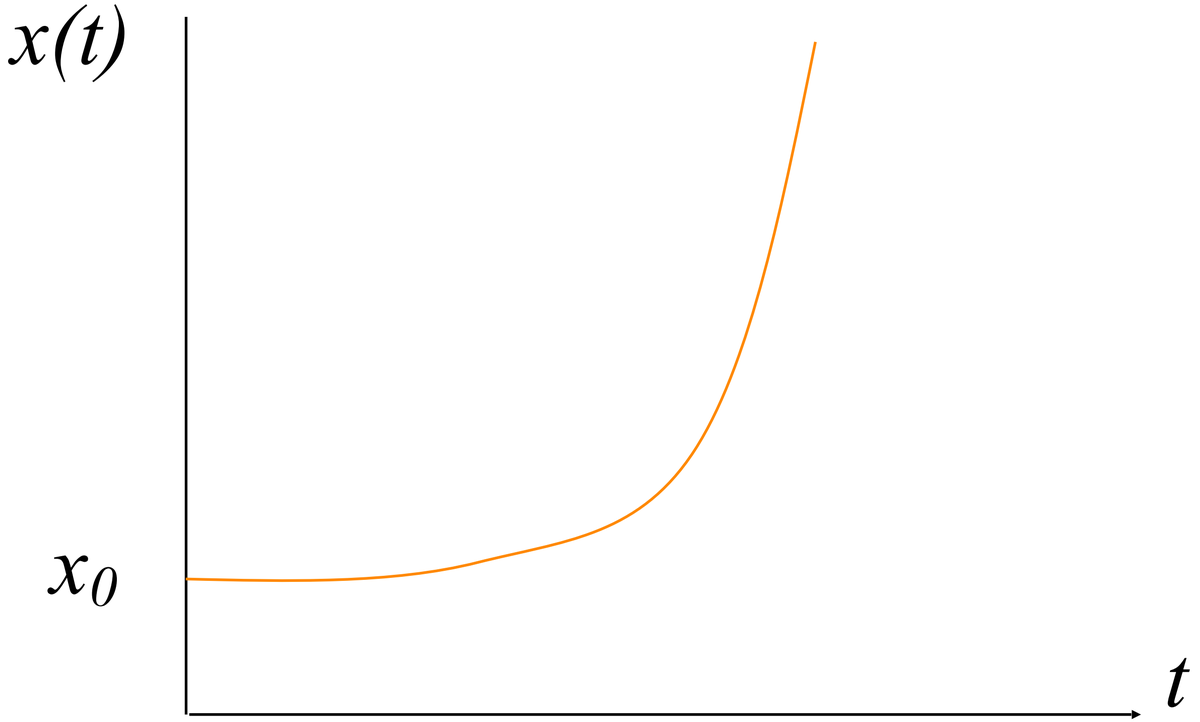
- To confirm this solution, substitute it into the original equation:

$$\frac{dx(t)}{dt} = ax(t)$$

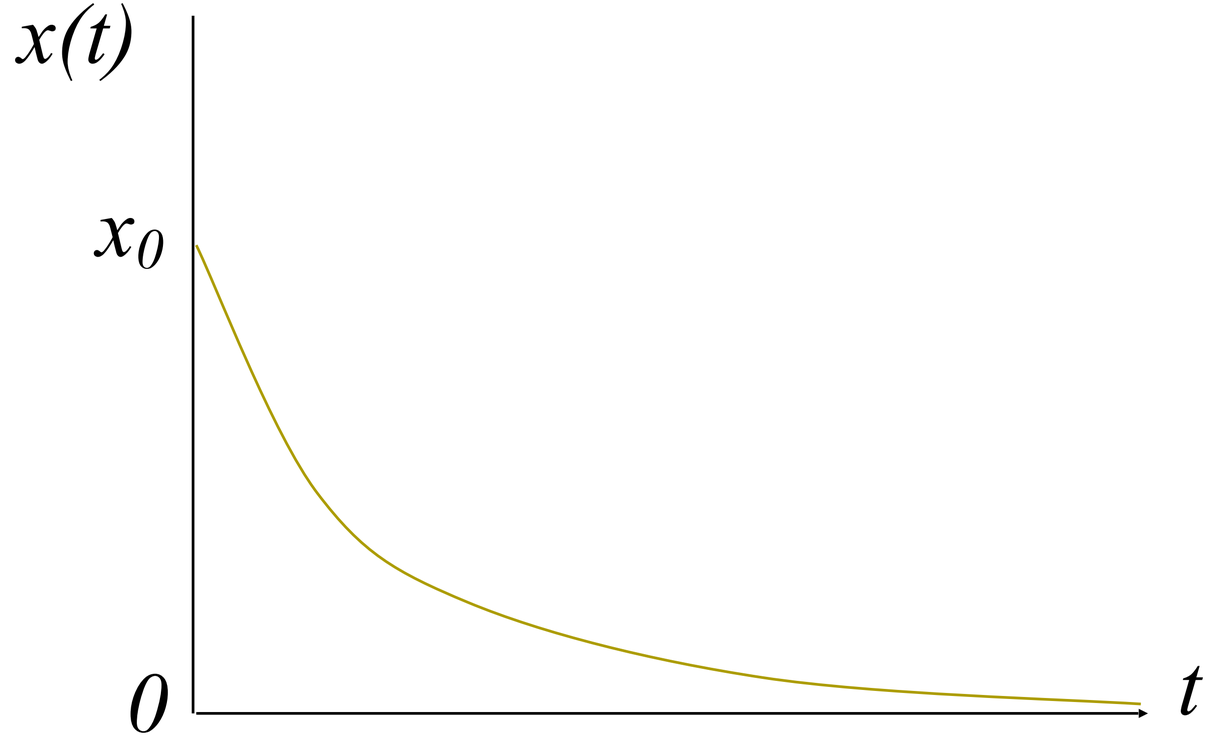
$$\frac{d[x_0 e^{at}]}{dt} = a[x_0 e^{at}]$$

$$ax_0 e^{at} = ax_0 e^{at}$$

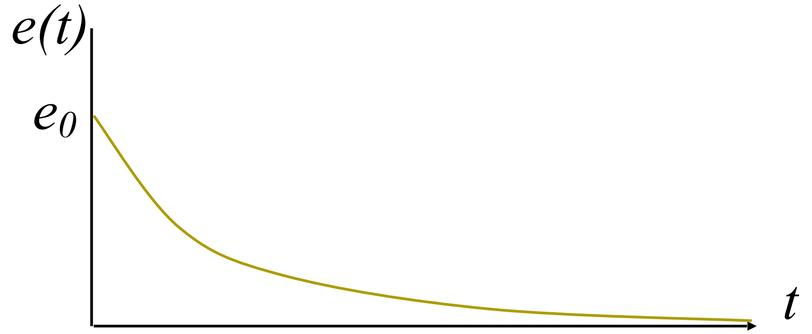
- To view how the state x behaves over time, we can plot out $x = x_0 \, exp(at)$, assuming a is positive:



- If a is negative and we can plot out $x=x_0\,exp(at)$, we get much different results:



- This exponential decay informs us that the state x decays to zero over time.
  - We say this system is “STABLE”.
  - We use this property in control theory to drive states down to zero (e.g. if $e = x_{desired} - x$, drive e to 0).



- The above example was a one dimensional linear system (i.e. single state x).
  - Our system is a multi-dimensional system (i.e. 3 states x, y, θ).
  - We need to describe the system with matrices:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t)$$

where $\mathbf{A}$ is a matrix such that $\mathbf{A} \in R^{nxn}$

$\mathbf{x}$ is a vector such that $\mathbf{x} \in R^{nx1}$

- The eigen values of $\mathbf{A}$, represented by λ_i, are coefficients that satisfy the equation:

$$\mathbf{A}\mathbf{x}_i(t) = \lambda_i \mathbf{x}_i(t)$$

for particular states called $\mathbf{x}_i$, called the eigen vectors.

- A solution to the system can be written as the combination of eigen vectors:

$$\mathbf{x}(t) = \mathbf{x}_1 e^{\lambda_1 t} + \mathbf{x}_2 e^{\lambda_2 t} + \ldots + \mathbf{x}_n e^{\lambda_n t}$$

- In this case, the system:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t)$$

is said to be stable if the eigen-values of $\mathbf{A}$ are less than 0.

- We solve for eigen values by noting:

$$(\mathbf{A} - \lambda_i \mathbf{I})\mathbf{x}_i(t) = 0$$

- Example:

$$A = \begin{bmatrix} 3 & 6 \\ 1 & 4 \end{bmatrix}.$$

- Summary:
  - If our robot behaves like a system of the form:

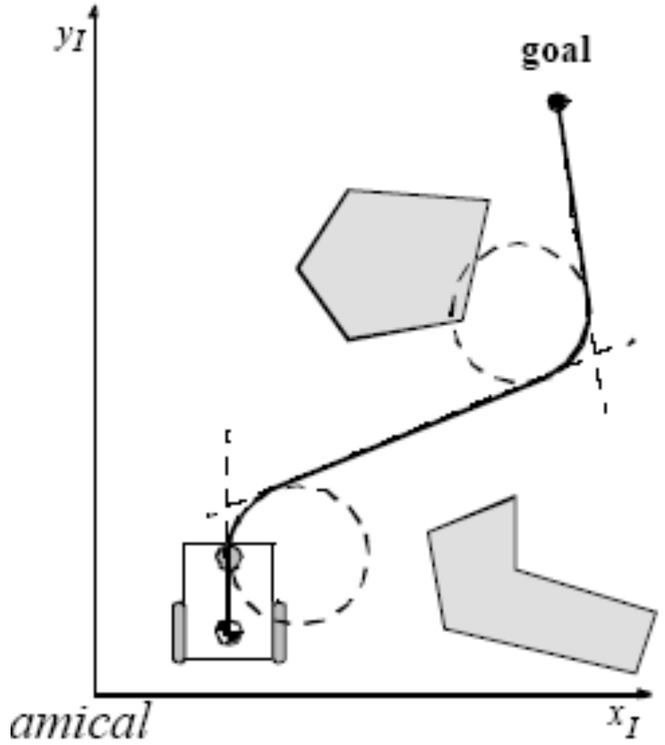
$$\dot{\mathbf{e}}(t) = \mathbf{A}\mathbf{e}(t)$$

  - where the eigen values of $\mathbf{A}$ are negative and $\mathbf{e}$ represents the difference between desired and actual states, the system will move to our desired state!

Agenda

1. Proportional Control
2. Linear Systems
3. **Point Tracking**

- Goal
  - Follow a trajectory from an initial state to some desired goal location.
- Several approaches
  - Could construct a global trajectory first, then track points on the trajectory locally

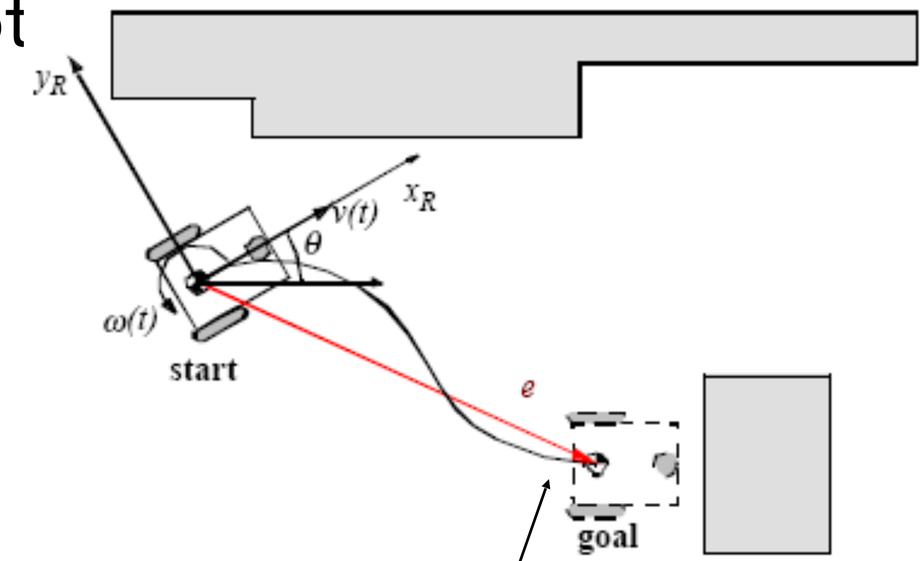


- If we define the error to be in the robot frame:

$$e(t) = [\, x \; y \; \theta \,]^T$$

- Goal is to find gain matrix K such that control of $v(t)$ and $w(t)$ will drive the error $e(t)$ to zero.

$$\begin{bmatrix} v(t) \\ w(t) \end{bmatrix} = Ke(t)$$



Assume goal is at [0 0 0]

- Recall our forward kinematics

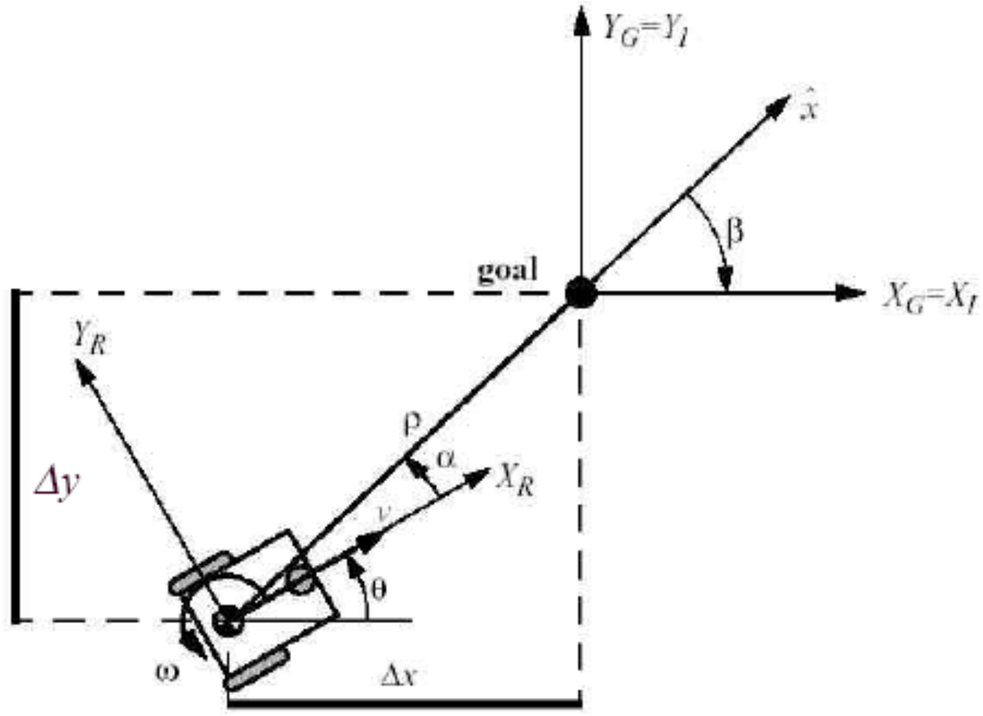
$$\begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{\theta}(t) \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v(t) \\ w(t) \end{bmatrix}$$

- We use the coordinate transformation

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}$$

$$\alpha = -\theta + atan2(\Delta y, \Delta x)$$

$$\beta = -\theta - \alpha$$



- Now we define the problem as driving the robot to goal

$$\begin{bmatrix} \rho \\ \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

- We know this will happen if the dynamics of the system obey

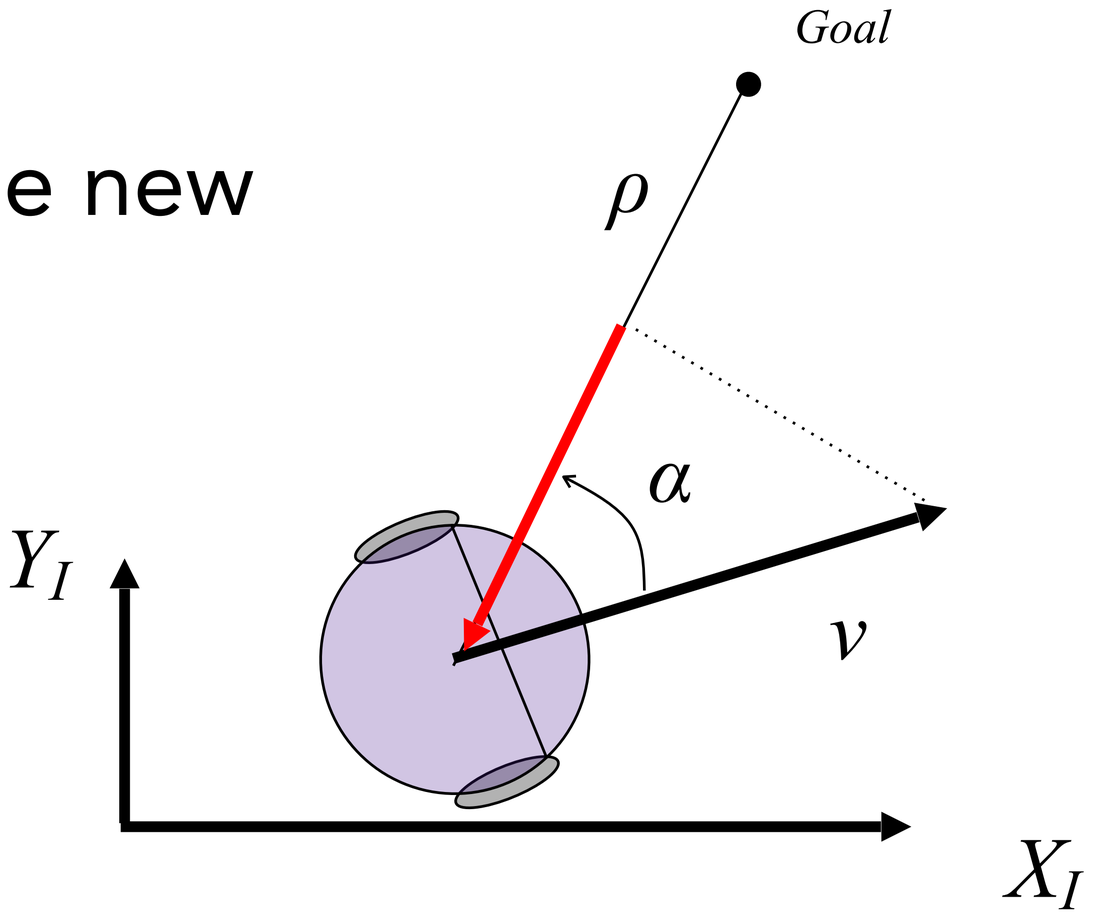
$$\begin{bmatrix} \dot{\rho}(t) \\ \dot{\alpha}(t) \\ \dot{\beta}(t) \end{bmatrix} = \mathbf{A} \begin{bmatrix} \rho(t) \\ \alpha(t) \\ \beta(t) \end{bmatrix}$$

Where $\mathbf{A}$ is a $3x3$ matrix with eigen values less than 0.

- Using the coordinate transformation, calculate the new kinematics:

$$\dot{\rho} =$$

$$\dot{\rho} = -v\cos\alpha$$

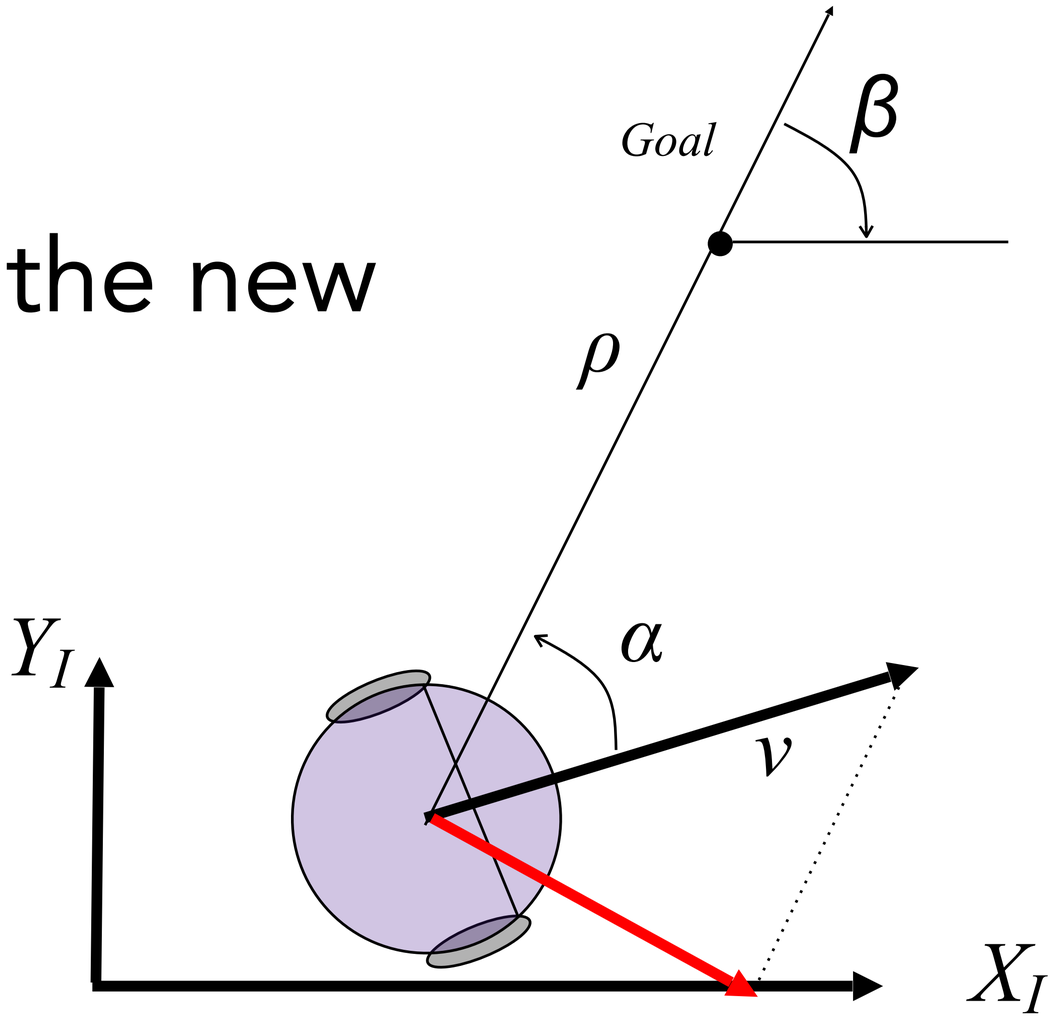


- Using the coordinate transformation, calculate the new kinematics:

$$\dot{\beta} =$$

$$\rho\dot{\beta} = -v\sin\alpha$$

$$\dot{\beta} = -v\sin\alpha/\rho$$

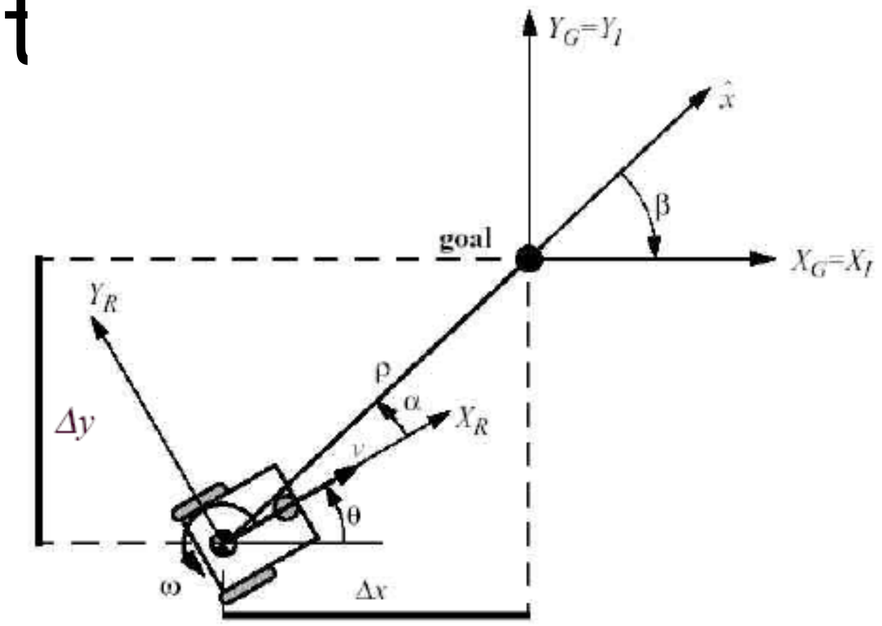


- Using the coordinate transformation, calculate the new kinemat˙

$$\alpha = -\beta - \theta$$

$$\dot{\alpha} = -\dot{\beta} - \dot{\theta}$$

$$\dot{\alpha} = v \sin\alpha / \rho - w$$



- In matrix from:

$$\begin{bmatrix} \dot{\rho}(t) \\ \dot{\alpha}(t) \\ \dot{\beta}(t) \end{bmatrix} = \begin{bmatrix} -\cos\alpha & 0 \\ \sin\alpha/\rho & -1 \\ -\sin\alpha/\rho & 0 \end{bmatrix} \begin{bmatrix} v(t) \\ w(t) \end{bmatrix}$$

for α within $(-\pi/2, \pi/2]$

- Let's try the control law:

$$v = k_\rho \rho \qquad w = k_\alpha \alpha + k_\beta \beta$$

- Note that this is a form of Proportional control, and if ρ, α, β all go to zero, then v and w will go to zero.

- To analyze controller, substitute control law into kinematics and linearize:

$$\begin{bmatrix} \dot{\rho}(t) \\ \dot{\alpha}(t) \\ \dot{\beta}(t) \end{bmatrix} = \begin{bmatrix} -k_\rho & 0 & 0 \\ 0 & -(k_\alpha - k_\rho) & -k_\beta \\ 0 & -k_\rho & 0 \end{bmatrix} \begin{bmatrix} \rho(t) \\ \alpha(t) \\ \beta(t) \end{bmatrix}$$

$$\dot{\mathbf{e}}(t) = \mathbf{A}\mathbf{e}(t)$$

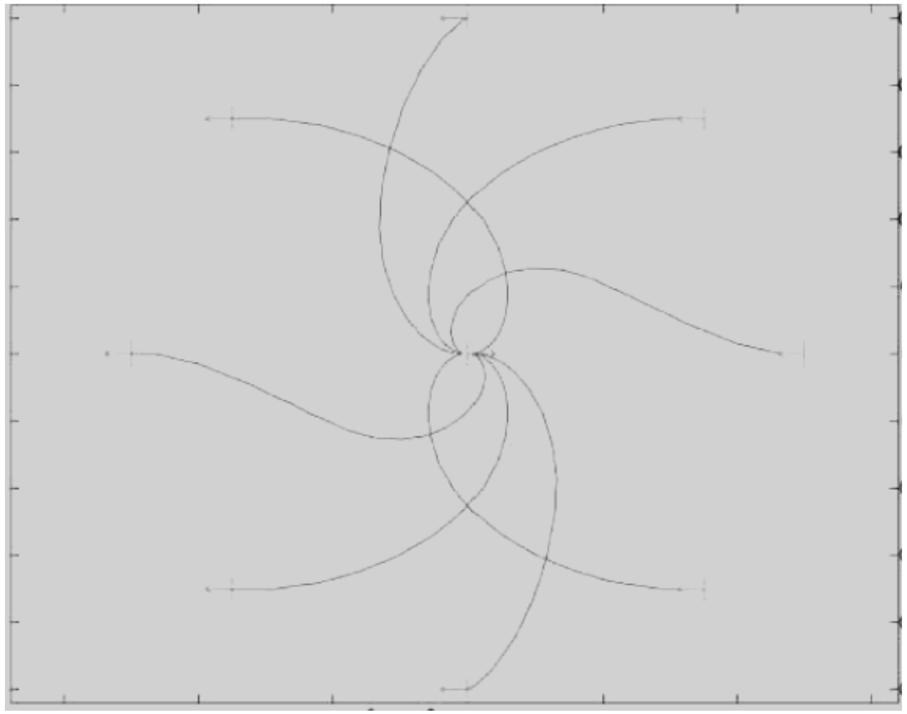
- Check for stability:
  - Take the determinant of $\mathbf{A}$ and solving for eigen values leads to:

$$(\lambda + k_\rho) (\lambda^2 + \lambda (k_\alpha - k_\rho) - k_\rho k_\beta) = 0$$

  - Thus the system will be stable if:

$$k_\rho > 0 \qquad k_\beta < 0 \qquad k_\alpha - k_\rho > 0$$

- Testing this control law with many different start points:

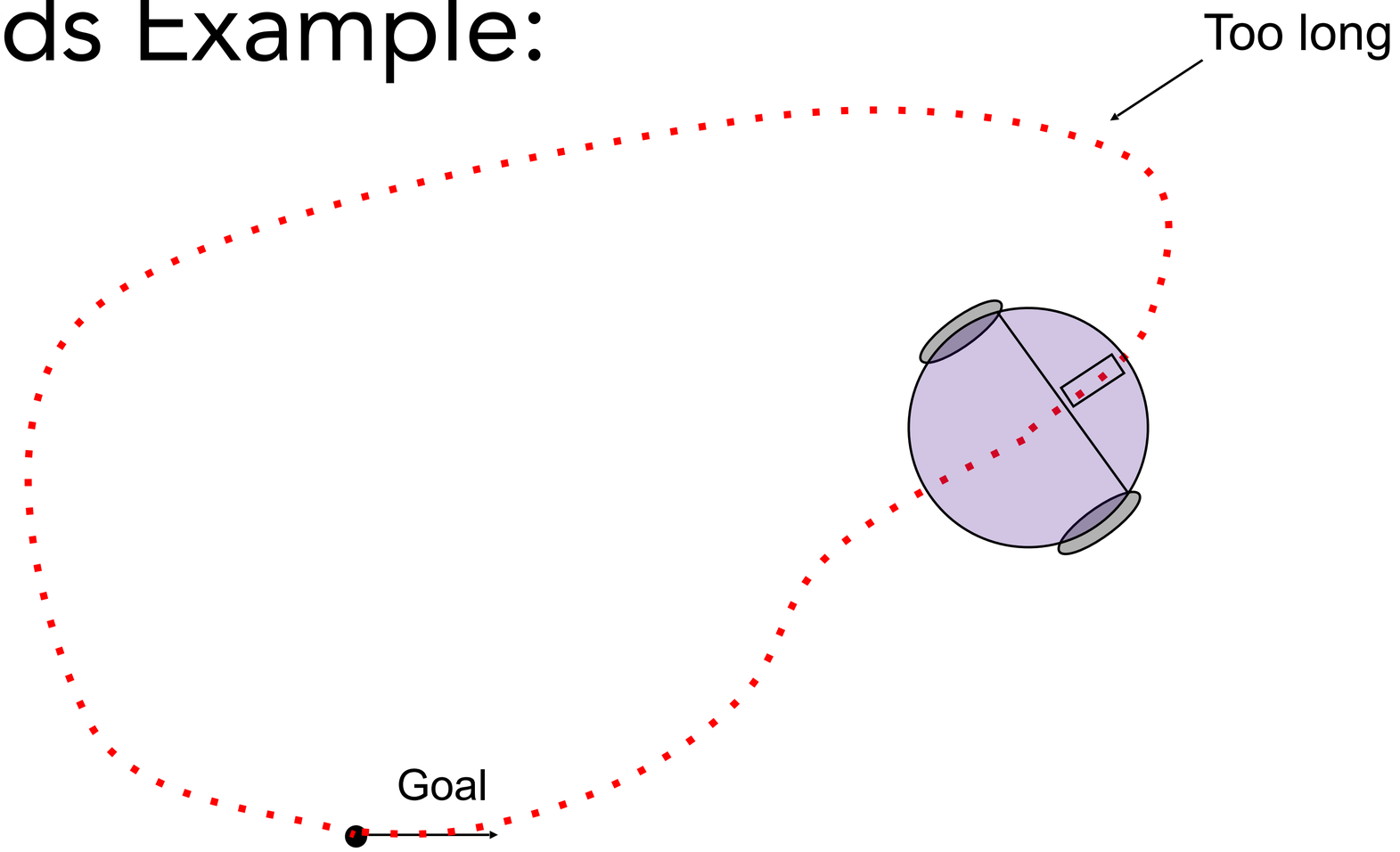


- The derived control law works well if

$$\alpha \in [-\pi/2, \pi/2]$$

  - For other cases where $abs(\alpha) > \pi/2$, we must modify the controller. So that the robot will move backwards to the desired position when required

- Backwards Example:

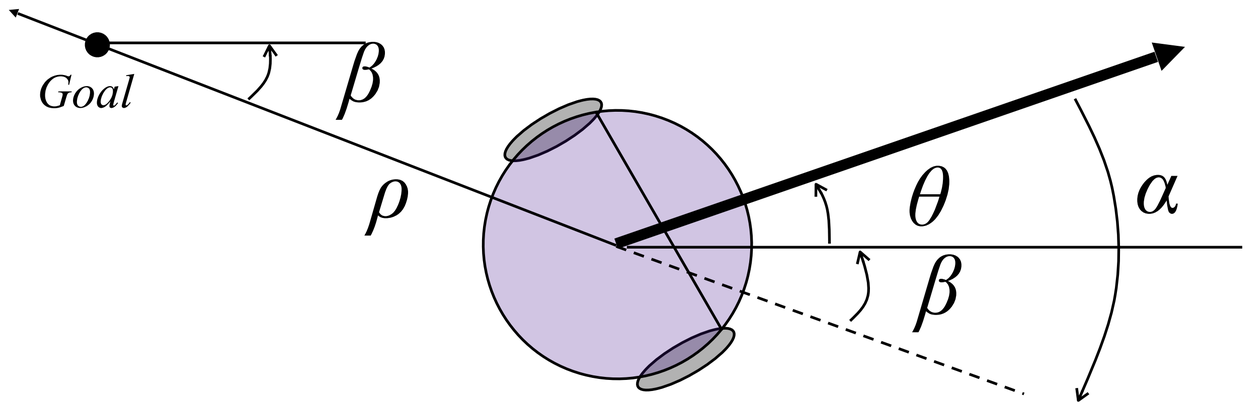


- Backwards Method:

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}$$

$$\alpha = -\theta + atan2(-\Delta y, -\Delta x)$$

$$\beta = -\theta - \alpha$$



- Backwards Method Summary:
  - If $\alpha \in [-\pi/2, \pi/2]$
    - Use regular transform to polar coordinates
    - Use control law: $v = k_\rho \rho \qquad w = k_\alpha \alpha + k_\beta \beta$
  - Else
    - Redefine α as shown in backwards method
    - Use control law: $v = -k_\rho \rho \qquad w = k_\alpha \alpha + k_\beta \beta$



ROB-UY 3303

Robot Motion & Planning



ROB-UY 3303

Robot Motion & Planning



ROB-UY 3303

Robot Motion & Planning